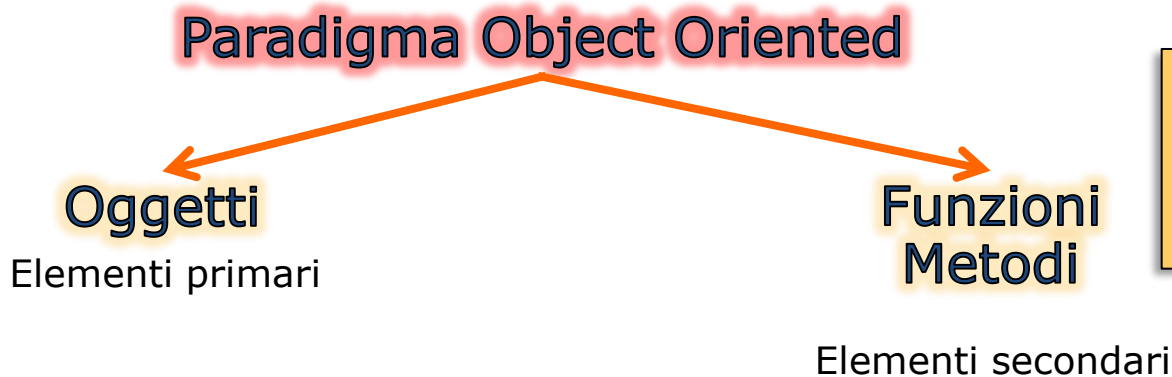


Linguaggio Java

Paradigma funzionale
e lambda expression

Programmazione funzionale

- Java storicamente nasce (1995) come linguaggio Object Oriented, in contrapposizione con il paradigma dominante del tempo, cioè il paradigma funzionale.
- La programmazione ad oggetti pone al centro dell'attenzione **l'oggetto come dato condiviso e mutabile.**
 - Elementi come metodi o classi sono considerati **accessori necessari** alla manipolazione degli oggetti, cioè **elementi secondari**



L'idea è quella di trattare le funzioni come elementi primari, come se fossero oggetti

Il concetto di predicato

- Supponiamo di voler filtrare tutti i file nascosti di una directory. Utilizzando i metodi della classe File si ottiene:

```
File[] hiddenFiles = new File(".").listFiles( new FileFilter() {  
    public boolean accept (File file) {  
        return file.isHidden();  
    }  
});
```

il codice (per quanto composto da sole 3 righe) è molto verboso se si considera di avere già metodi (ad alto livello) della classe File a disposizione

- Il problema è che in Java (7) i dati sono trattati come oggetti (elementi primari) e quindi anche **un predicato** deve essere trattato come oggetto
- **Java 8 introduce il concetto di funzione come parametro di un metodo**

Il concetto di predicato

- A ben vedere, il parametro del metodo `listFiles` è un pezzo di codice “camuffato” (wrappato) da oggetto Java

```
File[] hiddenFiles = new File(".").listFiles(  
    new FileFilter() {  
        public boolean accept (File file) {  
            return file.isHidden();  
        }  
    }  
);
```

funzione o predicato wrappato
in un oggetto Java

Necessario in quanto in Java
(7) i parametri di un metodo
possono essere solo oggetti o
primitivi

- In Java 8 la stessa operazione può essere fatta con il seguente codice:

```
File[] hiddenFiles = new File(".").listFiles((File f) -> f.isHidden());
```

comportamento passato come parametro di un metodo

Metodi come parametri di metodi

- Il concetto chiave è il seguente: **una funzione può essere passata come parametro di un metodo**
 - Si parla quindi di programmazione **secondo uno stile funzionale**
 - Ad un metodo è possibile passare primitivi, oggetti e funzioni
- Java 8 introduzione una nuova sintassi per le **espressioni lambda** allo scopo di passare funzioni come parametri di metodi



- I principali vantaggi sono i seguenti:
- migliore modularizzazione del codice
 - codice sintetico e leggibile
 - predisposizione al cambiamento dei requisiti

Ancora un esempio

- Supponiamo di voler scrivere un metodo che filtra su una collezione in base ad certo criterio: vogliamo filtrare mele in base al loro colore ed anche in base al loro peso. Si dovrebbero scrivere 2 metodi separati:

```
public static List<Mela> filtraMelePerColore(List<Mela> cassetta) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta) {  
        if(m.getColore().equals("verde")) listaFiltrata.add(m);  
    }  
    return listaFiltrata;  
}
```

```
public static List<Mela> filtraMelePerPeso(List<Mela> cassetta) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta) {  
        if(m.getPeso() > 150) listaFiltrata.add(m);  
    }  
    return listaFiltrata;  
}
```

Ancora un esempio

- Come si può notare dall'esempio, i due metodi si differenziano soltanto dalla **condizione** che determina se una mela deve essere aggiunta nella lista
 - In matematica, una tale condizione (che ritorna un boolean) **si chiama predicato**
- Mentre in Java 7 è necessario avere entrambi i metodi, la nuova **espressione lambda di Java 8** consente di passare un predicato come parametro di un metodo

Si avrebbe così un solo metodo la cui firma sarebbe:

```
public static List<Mela> filtraMele(List<Mela> cassetta, Predicate<Mela> p) {  
    ...  
}
```

invocabile secondo una nuova forma, ad esempio:

```
filtraMele(cassetta, (Mela m) -> m.getPeso() > 150 );
```

Il concetto di behavior parameterization

- Il problema del commesso:
 - Supponiamo di avere una persona che svolge il lavoro per conto nostro. Se facesse per noi solo la spesa al supermercato, dovremmo fornirgli solo la lista dei prodotti di acquistare



```
public void acquistaAlMarket(List<Prodotto> lista){
```

Prendi l'auto e guida fino al supermarket

Prendi un carrello ed entra

Prendi i prodotti (lista)

Paga alla cassa e metti i prodotti nei sacchetti

Metti i sacchetti in auto

Torna a casa

JAVA 7:
Stesso comportamento
(behavior)
dati diversi (object)

```
}
```


Il concetto di behavior parameterization

- Il problema del commesso:
 - Ma se volessimo una persona che svolge diversi lavori per conto nostro? Ad esempio, una volta va a fare la spesa un'altra vostra va a pagare una bolletta alla posta.



```
public void svolgiCompito (??){
```



JAVA 7: è possibile
simulare il concetto di
behavior
parameterization
attraverso il pattern
strategy

JAVA 8:
Diverso comportamento
(behavior)
Con diversi dati (object)

```
}
```

Procediamo per gradi

- Ritorniamo al problema delle mele:
 - Il concetto di parametrizzazione dei soli dati ci porterebbe ad avere:

```
public static List<Mela> filtraMelePerColore(List<Mela> cassetta) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta)  
        if(m.getColore().equals("verde")) listaFiltrata.add(m);  
    return listaFiltrata;  
}
```

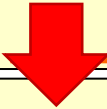


```
public static List<Mela> filtraMelePerColore(List<Mela> cassetta, String colore) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta)  
        if(m.getColore().equals(colore)) listaFiltrata.add(m);  
    return listaFiltrata;  
}  
  
List<Mela> meleVerdi = filtraMelePerColore(cassetta, "verde");
```

Procediamo per gradi

- Ritorniamo al problema delle mele:
 - Facciamo lo stesso per il peso

```
public static List<Mela> filtraMelePerPeso(List<Mela> cassetta) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta)  
        if(m.getPeso() > 150) listaFiltrata.add(m);  
    return listaFiltrata;  
}
```



```
public static List<Mela> filtraMelePerPeso(List<Mela> cassetta, int peso) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta)  
        if(m.getPeso() > peso) listaFiltrata.add(m);  
    return listaFiltrata;
```

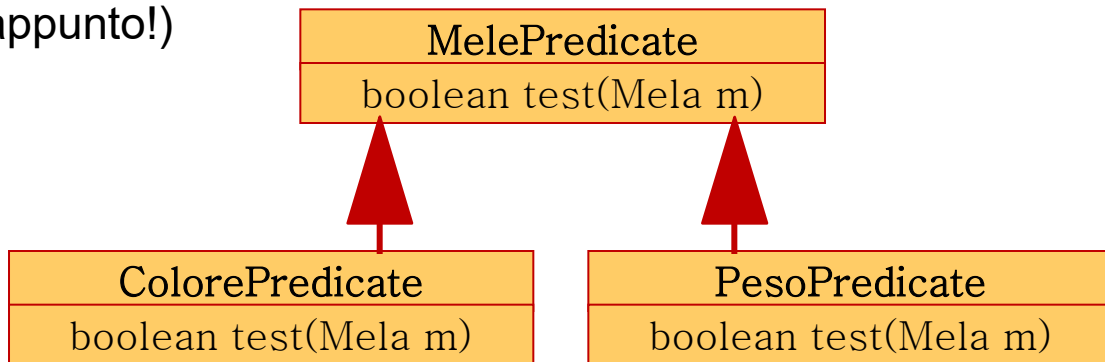
```
List<Mela> melePesanti = filtraMelePerPeso(cassetta, 150);
```

Procediamo per gradi: pattern Strategy

- Ma per parametrizzare il comportamento, cosa possiamo fare?
- Possiamo implementare il pattern strategy realizzando una interfaccia con una condizione testabile su un certo oggetto (quindi che ritorna un booleano)

```
public interface MelePredicate {  
    boolean test(Mela m);  
}
```

- Si utilizza per applicare una diversa strategia di comportamento (pattern strategy appunto!)



Procediamo per gradi: pattern Strategy

- Questo ci porta a scrivere un metodo generico che filtra sulle mele:

```
public static List<Mela> filtraMele(List<Mela> cassetta, MelePredicate p) {  
    List<Mela> listaFiltrata = new ArrayList<Mela>();  
    for(Mela m: cassetta)  
        if(p.test(m)) listaFiltrata.add(m);  
    return listaFiltrata;  
}
```

a patto di avere una o più classi di tipo MelePredicate (le diverse strategie)

```
public class ColorePredicate implemente MelePredicate {  
    public boolean test(Mela m) { return m.getColore().equals("verde"); }  
}
```

```
public class PesoPredicate implemente MelePredicate {  
    public boolean test(Mela m) {return m.getPeso() > 150; }  
}
```

Procediamo per gradi: inner class

- Per diminuire la verbosità, java introduce sin da subito le inner class anonime: elimina la necessità di creare in maniera esplicita classi di tipo Predicato

Invocazione esplicita:

```
MelePredicate p = new ColorePredicate();  
List<Mela> listaFiltrata = filtraMele(cassetta, p);
```

Invocazione via inner class anonima:

```
List<Mela> listaFiltrata = filtraMele(cassetta, new MelePredicate(){  
    public boolean test(Mela m) {  
        return m.getColore().equals("verde");  
    }  
});
```

Procediamo per gradi: espressioni lambda

- Java 8 attraverso le espressioni lambda ingegnerizza e riorganizza questo scenario

Invocazione esplicita (Java 7):

```
MelePredicate p = new ColorePredicate();  
List<Mela> listaFiltrata = filtraMele(cassetta, p);
```

Invocazione via inner class anonima (Java 7):

```
List<Mela> listaFiltrata = filtraMele(cassetta, new MelePredicate(){  
    public boolean test(Mela m) {  
        return m.getColore().equals("blu");  
    }  
});
```

Invocazione via espressione lambda (Java 8):

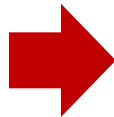
Nuovo operatore ->

```
List<Mela> lista = filtraMele(cassetta, (Mela m) -> m.getColore().equals("blu"));
```

Procediamo per gradi: generics

- Per migliorare ulteriormente lo schema si potrebbero utilizzare i generics. L'interfaccia MelePredicate, si potrebbe definire come generica interfaccia che rappresenta un predicato per poi parametrizzarla via generics

```
public interface MelePredicate {  
    boolean test(Mela m);  
}
```



```
public interface Predicate<T> {  
    boolean test(T m);  
}
```

Java 8

- Java 8 introduce tale interfaccia nel package **java.util.function**.

```
public static <T> List<T> filtra(List<T> lista, Predicate<T> p) {  
    List<T> listaFiltrata = new ArrayList<>();  
    for(T t: lista)  
        if(p.test(t)) listaFiltrata.add(t);  
    return listaFiltrata;  
}
```

Utilizzo dei generics a livello di metodo

Ricapitoliamo

- Per sfruttare a pieno le nuove possibilità abbiamo bisogno di 3 elementi:
 - 1. Functional interface** → l'interfaccia che rappresenta un predicato (come `Predicate<T>`)
 - 2. Behavior parameterization** → metodo che ha come argomento un comportamento. Cioè un metodo che prende come argomento un predicato
 - 3. Lambda expression** → nuova sintassi per passare una funzione in sostituzione dell'oggetto predicato

```
1 public interface Predicate<T> {  
    boolean test(T m);  
}  
2 public static void esegui(String obj, Predicate<String> p) {  
    System.out.println(p.test(obj));  
}  
3 esegui("ciao", (String s) -> s.length() >3);
```

JAVA 8:
Diverso comportamento (behavior)
Con diversi dati (object)

dato

comportamento

Espressioni lambda

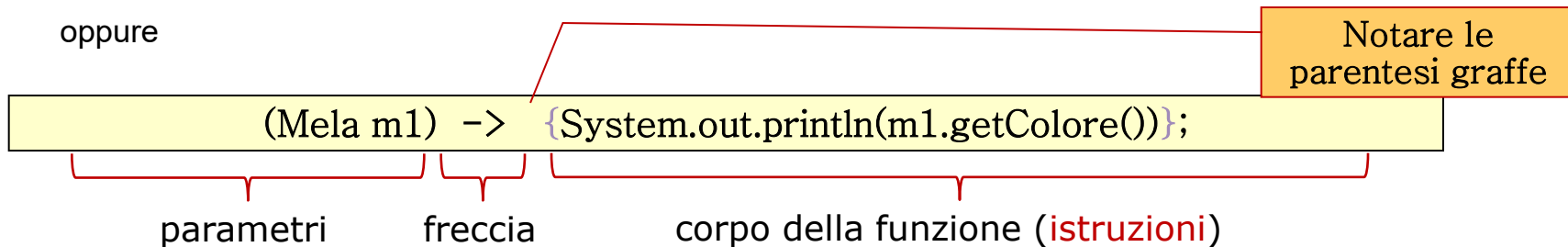
- Le espressioni lambda sono:
 - **Funzioni** → sono funzioni perché il codice non si riferisce ad una particolare classe come accade ai normali metodi
 - **Anonime** → sono funzioni prive di nome
 - **Passabili** → si possono passare ad un metodo come normali argomenti
 - **Concise** → sintassi non verbosa al contrario di quello che accade con le inner class anonime
- **Nota:** le espressioni lambda non introducono nuove caratteristiche al linguaggio Java; si tratta più che altro di un nuovo stile molto più flessibile e pulito
- **Nota:** Java evita di introdurre il tipo funzione. Piuttosto utilizza le **interfacce funzionali**.

Sintassi espressione lambda

- In generale la sintassi si compone di 3 parti distinte



oppure



La differenza sta nel tipo di ritorno del metodo dell'interfaccia di tipo Predicate

- Corpo della funzione **espressione** → metodo ritorna qualcosa (non void)
- Corpo della funzione **istruzioni** → metodo ritorna void

Sintassi espressione lambda

- Una interfaccia Funzionale (come Predicate) regola l'invocazione stile lambda e deve seguire alcune regole sintattiche:
 - Deve essere una interfaccia
 - Può utilizzare i generics
 - Deve avere esattamente un solo metodo
 - Il metodo può ritornare qualsiasi cosa, può avere qualsiasi parametro o sollevare qualsiasi eccezione
- Esempi

```
public interface A {boolean test(Mela m);}
```

```
}
```

```
(Mela m) → m.getColor().equals("red")
```

```
public interface B {boolean test();}
```

```
}
```

```
() → new Random().nextInt(100)>50
```

```
public interface C {void test(Mela m);}
```

```
}
```

```
(Mela m) → { System.out.println(m); }
```

```
(Mela m) → System.out.println(m) Singola riga
```

```
public interface D {int test(String s);}
```

```
}
```

```
(String s) → s.length()
```

```
public interface E {int test(int a, int b);}
```

```
}
```

```
(int v1, int v2) → v1* v2
```

Sintassi espressione lambda: assegnazioni

- Secondo quanto appreso, anche la seguente assegnazione è valida:

```
public interface Runnable {  
    void run();  
}
```

Le parentesi bisogna scriverle vuote perché il metodo run() non prende parametri

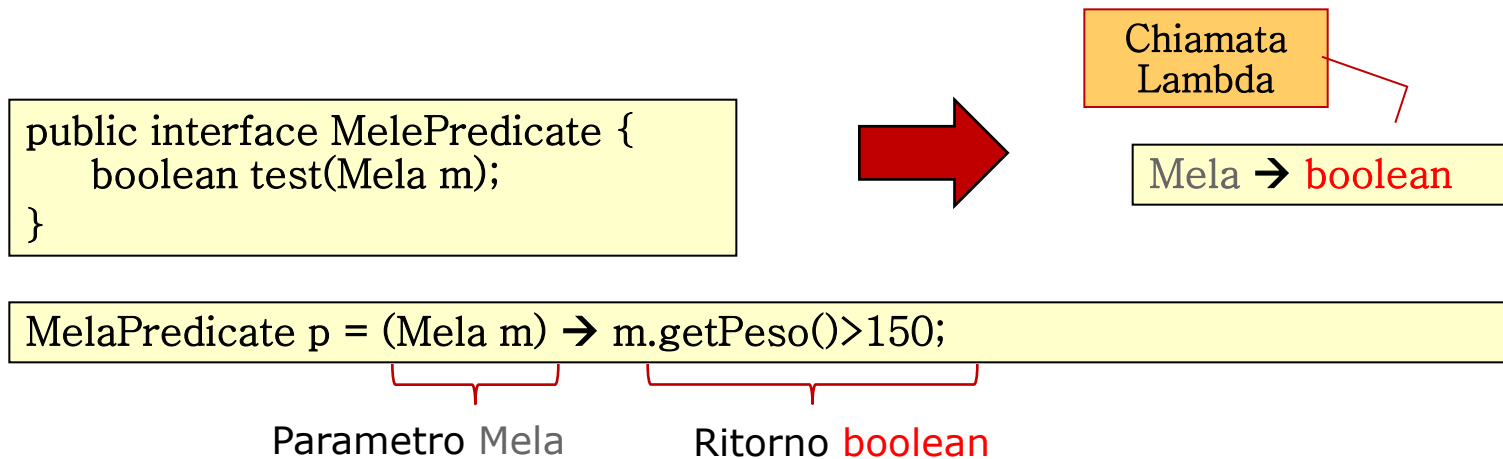
```
Runnable r = () → System.out.println("funziona!"); //assegnazione lambda  
Thread t = new Thread(r);  
t.start();
```

- Java, anche prima della versione 8, offriva già diverse interfacce funzionali come ad esempio:
 - Runnable
 - Comparator<T>
 - Callable<V>

e molte altre...

Sintassi espressione lambda: type checked

- Le espressioni lambda sono **type checked**, cioè il compilatore è in grado di controllare la validità dei tipi all'interno delle espressioni.
- Ad esempio, supponiamo di avere l'interfaccia funzionale MelaPredicate, il cui metodo prende un oggetto Mela e ritorna un **boolean**:



Sintassi espressione lambda: type inference

- E' possibile semplificare ancora un po' la sintassi sfruttando il **type inference** del compilatore.
 - Se il compilatore è in grado di determinare il tipo del parametro è possibile ometterlo nella espressione lambda.

```
public interface Predicate<T> {  
    boolean test(T m);  
}
```

```
public static void esegui(String obj, Predicate<String> p) {  
    System.out.println(p.test(obj));  
}
```

```
esegui("ciao", (String s) -> s.length() > 3);
```

```
esegui("ciao", s -> s.length() > 3);
```

Tipo dedotto in automatico dal compilatore.

EQUIVALENTI

@FunctionalInterface

- Java 8 introduce una annotazione per marcare le interfacce funzionali: @FunctionalInterface
- L'annotazione è facoltativa e serve ad indicare che una certa interfaccia è da considerarsi una interfaccia funzionale
 - In questo caso, il compilatore controllerà le regole sintattiche per le interfacce funzionali come ad esempio la presenza di un unico metodo
- Dal punto di vista pratico, svolge lo stesso ruolo della annotazione @Override
 - Come @Override, è buona pratica utilizzarla

Le interfacce funzionali di Java 8

- Java 8 fornisce le seguenti interfacce funzionali

Interfaccia	Chiamata lambda	Specializzazioni
Predicate<T>	T -> boolean	IntPredicate, LongPredicate, DoublePredicate
Consumer<T>	T -> void	IntConsumer, LongConsumer, DoubleConsumer
Function<T, R>	T -> R	IntFunction<R>, IntToDoubleFunction, IntToLongFunction, LongFunction<R>, LongToDoubleFunction, LongToIntFunction, DoubleFunction<R>, ToIntFunction<T>, ToDoubleFunction<T>, ToLongFunction<T>

Le interfacce funzionali di Java 8

Interfaccia	Chiamata lambda	Specializzazioni
Supplier<T>	() -> T	BooleanSupplier, IntSupplier, LongSupplier, DoubleSupplier
UnaryOperator<T>	T -> T	IntUnaryOperator, LongUnaryOperator, DoubleUnaryOperator
BinaryOperator<T>	(T, T) -> T	IntBinaryOperator, LongBinaryOperator, DoubleBinaryOperator
BiPredicate<L, R>	(L, R) -> boolean	BiConsumer<T, U> (T, U) -> void ObjIntConsumer<T>, ObjLongConsumer<T>, ObjDoubleConsumer<T>
BiFunction<T, U, R>	(T, U) -> R	ToIntBiFunction<T, U>, ToLongBiFunction<T, U>, ToDoubleBiFunction<T, U>

Le interfacce funzionali di Java 8

- Notare che le interfacce funzionali offerte da Java 8 utilizzano i generics per aumentare le possibilità di utilizzo.
- L'uso dei generics però esclude i primitivi che possono essere utilizzati solo attraverso i wrapper Java (con o senza autoboxing)
 - L'autoboxing usa implicitamente i wrapper
- Usare i wrapper esplicitamente o usare la tecnica del boxing ha un costo computazionale che potrebbe essere non trascurabile.
- Per evitare l'uso dei wrapper, Java offre interfaccia come specializzazione delle principali che contemplano l'uso diretto dei primitivi

Predicate<T>	IntPredicate, LongPredicate, DoublePredicate
---------------------------	---

Espressioni lambda: scope variabili

- Le espressioni lambda sono equivalente alle inner class anonime, e dunque sono soggette alle stesse restrizioni sulle variabili

free variables: variabili dichiarate fuori dallo scope della inner class anonima

```
public static void main(String[] args){  
    int x = 10;  
    List<Mela> listaFiltrata = filtraMele(cassetta, new MelePredicate(){  
        public boolean test(Mela m) {  
            return m.getPeso() > x;  
        }  
    }); }  
}
```

```
public static void main(String[] args){  
    int x = 10; // free variable soggetta a restrizioni  
    List<Mela> lista = filtraMele(cassetta, (Mela m) -> m.getPeso() > x);  
}
```

Espressioni lambda: scope variabili

- Le free variable sono soggette queste regole:

NESSUNA RESTRIZIONE

- Variabili di istanza
- Variabili statiche

FINAL o non riassegnabile

- Variabili locali

una variabile locale può anche non essere final ma va trattata come tale:
sono vietate le riassegnazioni

```
public static void main(String[] args){  
    int x = 10; // free variable soggetta a restrizioni  
    List<Mela> lista = filtraMele(cassetta, (Mela m) -> m.getPeso() > x);  
    x = 20; //errore di compilazione: la variabile non è riassegnabile  
}
```

Method reference

- La sintassi delle espressioni lambda è ulteriormente semplificata se nel implementare il predicato si invoca un solo metodo. Ad esempio:

```
File[] hiddenFiles = new File(".").listFiles(  
    new FileFilter() {  
        public boolean accept (File file) {  
            return file.isHidden();  
        }  
    }  
);
```

funzione o predicato wrappato
in un oggetto Java

FileFilter è quindi una
interfaccia funzionale, cioè un
predicato

**Il predicato chiama il solo
metodo isHidden()**

listFiles() prende il predicato FileFilter e può quindi essere invocato così:

```
File[] hiddenFiles = new File(".").listFiles((File f) -> f.isHidden());
```

ma anche più sinteticamente così:

```
File[] hiddenFiles = new File(".").listFiles(File::isHidden);
```

Method reference

- E' possibile utilizzare la tecnica del **method reference** in questi 3 casi:
 - Riferimento ad un metodo statico
 - Riferimento ad un metodo di istanza attraverso un parametro
 - Riferimento ad un metodo di istanza attraverso un oggetto esistente
- Mentre il primo caso è relativamente semplice, gli altri 2 sono più complessi da comprendere

NOTA: la tecnica del method reference è una semplificazione della sintassi della invocazione lambda e non offre nuove possibilità

Method reference: metodo statico

- Caso in cui nel predicato si invoca un metodo statico. Ad esempio, supponiamo di avere il seguente metodo che prendere un predicato Function:

```
public static void test(String s, Function<String, Integer> f ){  
    System.out.println("risultato: " + f.apply(s));  
}
```

la chiamata lambda è **(String → Integer)**; la normale invocazione lambda:

```
test("1234", s -> Integer.parseInt(s)); //invocazione del solo metodo parseInt
```

l'invocazione con la tecnica del method reference :

```
test("1234", Integer::parseInt);
```

- In questo caso il compilatore riconosce parseInt come un metodo statico della classe Integer; per rispettare la chiamata lambda si aspetta che parseInt prenda come parametro una stringa e torni un intero.

(String → Integer)



static int parseInt(String s)

Method reference: metodo statico

- Altro caso in cui il predicato invoca un metodo statico. Ad esempio, supponiamo di avere il seguente metodo che prendere un BiFunction:

```
public static void test(int a, int b, BiFunction<Integer, Integer, Integer> b ){  
    System.out.println("risultato: " + b.apply(a, b));  
}
```

la chiamata lambda è ((Integer, Integer) → Integer); invocazione normale

```
test(2, 4, (i1,i2) -> Math.max(i1,i2)); //invocazione del solo metodo max
```

l'invocazione con la tecnica del method reference :

```
test(2,3, Math::max);
```

- In questo caso il compilatore riconosce max come un metodo statico della classe Math; per rispettare la chiamata lambda si aspetta che max prenda come parametro due interni e torni un intero.

((Integer, Integer) → Integer)



static int max(int a, int b)

Method reference: metodo statico

- Regole del compilatore:
 - La logica del compilatore è quella di far corrispondere i parametri del metodo con la chiamata lambda.

`((Integer, Integer) → Integer)`



`static int max(int a, int b)`

- Le seguenti assegnazioni sono equivalenti:

```
BiFunction<Integer, Integer, Integer> b = (i1,i2) -> Math.max(i1,i2);
```

```
BiFunction<Integer, Integer, Integer> b = Math::max;
```

```
System.out.println(b.apply(1,2)); // così si invoca: Math.max(1,2);
```

- Le interfacce funzionali offerte dalla libreria Java non vanno oltre i 2 parametri (BiPredicate e BiFunction). Se si vuole invocare un metodo con 3 o più parametri bisogna scrivere una propria interfaccia funzionale

Method reference: metodo di istanza su parametro

- Caso in cui il predicato invoca un metodo non statico. Ad esempio, supponiamo di avere il seguente metodo che prendere un BiPredicate:

```
public static void test(String s, String s2, BiPredicate<String, String> p ){  
    System.out.println(p.test(s, s2));  
}
```

la chiamata lambda è **(String, String → boolean)**; la normale invocazione:

```
test("ciao", "c", (s1, s2) -> s1.startsWith(s2)); //invocazione del solo startWith
```

l'invocazione con la tecnica del method reference :

```
test("1234", "c", String::startsWith);
```

- In questo caso il compilatore riconosce `startsWith` come un metodo di istanza della classe `String`; per rispettare la chiamata lambda si aspetta che `startsWith` sia invocabile su un oggetto `String`, prenda come parametro una stringa e torni un `boolean`.

(String, String → boolean)



boolean s.startsWith(String s)

Method reference: metodo di istanza su parametro

- Regole del compilatore:
 - Anche in questo caso, la logica del compilatore è quella di far corrispondere i parametri del metodo con la chiamata lambda.

(String, String → boolean)



boolean s.startsWith(String s)

- Le seguenti assegnazioni sono equivalenti:

```
BiPredicate<String, String> b = (s1, s2) -> s1.startsWith(s2);
```

```
BiPredicate<String, String> b = String::startsWith;
```

- In questo caso però, l'interfaccia funzionale deve avere almeno un parametro di input che verrà utilizzato come istanza su cui invocare il metodo

```
BiPredicate<String, String> b = String::startsWith;
```

```
System.out.println(b.test("ciao", "c")); // così si invoca: "ciao".startsWith("c");
```

Method reference: metodo di istanza su oggetto esistente

- Caso molto simile al precedente. Ad esempio, supponiamo di avere il seguente metodo che prendere un Predicate:

```
public static void test(String s, Predicate<String> p ){  
    System.out.println(p.test(s));  
}
```

la chiamata lambda è (String → boolean); la normale invocazione:

```
test("c", s -> "ciao".startsWith(s)); //invocazione del solo startWith
```

l'invocazione con la tecnica del method reference :

```
test("c", "ciao"::startsWith);
```

- Come nel caso precedente, il compilatore riconosce startWith come un metodo di istanza della classe String; per rispettare la chiamata lambda si aspetta che startWith sia invocabile su un oggetto String; in questo caso l'oggetto non è un parametro ma un oggetto esistente

(String → boolean)

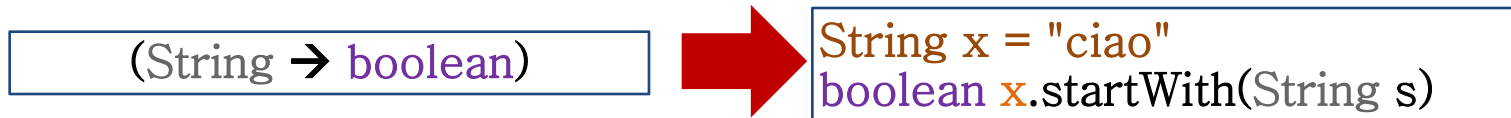


boolean "ciao".startsWith(String s)

Method reference: metodo di istanza su oggetto esistente

- Regole del compilatore:

- Anche in questo caso, la logica del compilatore è quella di far corrispondere i parametri del metodo con la chiamata lambda.



- Le seguenti assegnazioni sono equivalenti:

```
String x = "ciao"; Predicate<String> p = s -> x.startsWith(s);
```

```
Predicate<String> p = x::startsWith;
```

l'interfaccia funzionale può non avere parametri se il metodo usato nel predicato non prende parametri (vedi prox esempio)

```
String x = "ciao";
```

```
Predicate<String> p = x::startsWith;
```

```
System.out.println(p.test("c")); // così si invoca: "ciao".startsWith("c");
```

Method reference: metodo di istanza su oggetto esistente

- Esempio che utilizza l'interfaccia funzionale Supplier che non ha parametri di input

`() → Integer`



```
String x = "ciao";  
int x.length()
```

- Le seguenti assegnazioni sono equivalenti:
- In questo caso, l'interfaccia funzionale **NON** deve avere nessun parametro di input (infatti Supplier non ha parametri)

```
String x = "ciao";  
Supplier<Integer> p = () -> x.length();  
Supplier<Integer> p = x::length;
```

```
String x = "ciao";  
Supplier<Integer> p = x::length;  
System.out.println(p.get()); // così si invoca: "ciao".length();
```

Method reference: costruttore

- Un caso particolare è l'invocazione al costruttore. Supponiamo di avere il seguente metodo che prendere un Supplier:

```
public static void test(Supplier<java.util.Date> p ){  
    System.out.println(p.get());  
}
```

la chiamata lambda è `() -> java.util.Date`; la normale invocazione:

```
test(() -> new java.util.Date()); //invocazione al costruttore della classe java.util.Date
```

in questo caso, il metodo (unico) invocato è il costruttore. l'invocazione con la tecnica del method reference :

```
test(java.util.Date::new);
```

il costruttore è accomunabile ad un metodo statico. In questo caso il costruttore non deve avere parametri

- Se si utilizza una interfaccia funzionale che prevede parametri, questi diventano i parametri del costruttore

Comporre lambda expressions

- Molte delle interfacce funzionali offerte da Java 8 offrono metodi grazie ai quali è possibile comporre diverse espressioni lambda
 - Si tratta di metodi concreti presenti nelle interfacce: una nuova possibilità offerta da Java 8.
- Consideriamo l'interfaccia funzionale Comparator

`((String, String) → int)`



`int s1.compareTo(String s2)`

```
Comparator<String> com = (s1,s2) -> s1.compareTo(s2);  
Comparator<String> com = String::compareTo;
```

```
com = com.reversed(); //inversione della condizione
```

- Su un oggetto Comparator possono essere invocati diversi metodi della interfaccia Comparator. Ad esempio:

Principali metodi di composizione

- Altro esempio con l'interfaccia Predicate

```
Predicate<String> p1 = s -> s.length()>3;  
Predicate<String> p2 = s -> s.contains("x");  
Predicate<String> p3 = p1.and(p2);  
Predicate<String> p4 = p1.or(p2);  
Predicate<String> p5 = p1.negate();  
  
System.out.println(p3.test("ciao")); //stampa false  
System.out.println(p4.test("ciao")); //stampa true  
System.out.println(p5.test("ciao")); //stampa false
```

i metodi `and`, `or` e `negate` rappresentano la classica logica di AND, OR e NOT tra le condizioni

- L'interfaccia `Predicate` rappresenta condizioni

Principali metodi di composizione

- Altro esempio con l'interfaccia Function

```
Function<Integer, Integer> f = i -> i+ 1;  
Function<Integer, Integer> g = i -> i*2;  
  
Function<Integer, Integer> f3 = f.andThen(g);           // g(f())  
Function<Integer, Integer> f4 = f.compose(g);           // f(g())  
  
System.out.println(f3.apply(1));           //stampa 4  
System.out.println(f4.apply(1));           //stampa 3
```

i metodi andThen e compose rappresentano la logica matematica di applicazioni composite delle funzioni: $f(g())$ oppure $g(f())$